

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №1
По дисциплине «Интеллектуальный анализ данных»
Тема: «РСА»

Выполнил:
Студент 4 курса
Факультета ЭИС
Группы ИИ-26
Сугак В.А.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: научиться применять метод PCA для осуществления визуализации данных.

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);

2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;

3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;

4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Класс
17	hcv+data.zip	Category

Ход работы

Код программы:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

columns = [
    "ID", "Category", "Age", "Sex", "ALB", "ALP",
    "ALT", "AST", "BIL", "CHE", "CHOL", "CREA",
    "GGT", "PROT"
]

data = pd.read_csv("1.csv", header=None, names=["raw"])
data = data["raw"].str.split(",", expand=True)
data.columns = columns
data = data.iloc[1:].copy()

features = [
    "Age", "ALB", "ALP", "ALT", "AST",
    "BIL", "CHE", "CHOL", "CREA", "GGT", "PROT"
]

X = data[features].apply(pd.to_numeric, errors="coerce")
y = data["Category"].str.replace(' ', '')

valid = X.notna().all(axis=1)
X = X[valid]
```

```
y = y[valid]

X_std = (X - X.mean()) / X.std()

cov_matrix = np.cov(X_std, rowvar=False)

eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

idx = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]

X_pca_manual = X_std.values @ eigenvectors[:, :3]

explained_variance = eigenvalues / eigenvalues.sum()

print("Размер исходных данных:", X.shape)

print("\nСобственные значения:")
print(eigenvalues)

print("\nДоля объяснённой дисперсии:")
print(explained_variance)

print("\nРазмерность после PCA:", X_pca_manual.shape)

pca = PCA(n_components=3)
X_pca_sklearn = pca.fit_transform(X_std)

print("\nPCA sklearn:")
print("Собственные значения:")
print(pca.explained_variance_)

print("\nДоля объяснённой дисперсии:")
print(pca.explained_variance_ratio_)

loss_2 = 1 - pca.explained_variance_ratio_[ :2].sum()
loss_3 = 1 - pca.explained_variance_ratio_[ :3].sum()

print("\nПотери при использовании 2 компонент:", loss_2)
print("Потери при использовании 3 компонент:", loss_3)

classes = y.unique()
```

```

plt.figure(figsize=(9, 6))

for cls in classes:
    mask = y == cls
    plt.scatter(
        X_pca_sklearn[mask, 0],
        X_pca_sklearn[mask, 1],
        label=cls,
        alpha=0.7
    )

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Первые две главные компоненты")
plt.legend()
plt.grid()
plt.show()

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection="3d")

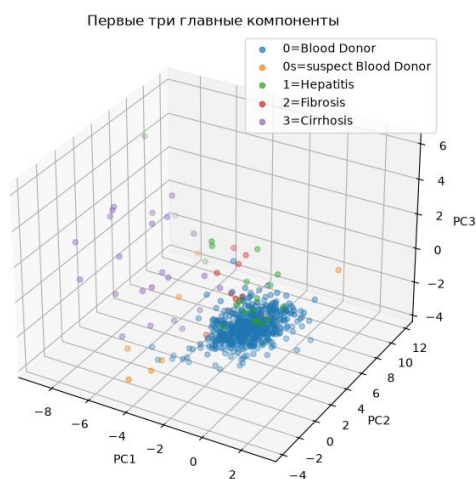
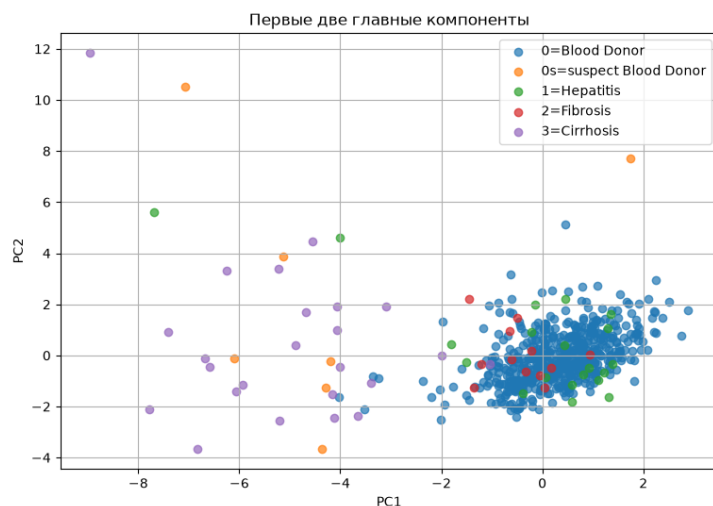
for cls in classes:
    mask = y == cls
    ax.scatter(
        X_pca_sklearn[mask, 0],
        X_pca_sklearn[mask, 1],
        X_pca_sklearn[mask, 2],
        label=cls,
        alpha=0.7
    )

ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title("Первые три главные компоненты")
ax.legend()

plt.show()

```

Графики:



Вывод программы:

```
Размер исходных данных: (589, 11)

Собственные значения:
[2.45450576 1.85655376 1.35296667 1.08245876 1.02714555 0.73377431
 0.66955793 0.58231356 0.5002047 0.42301113 0.31750788]

Доля объяснённой дисперсии:
[0.22313689 0.16877761 0.12299697 0.09840534 0.09337687 0.06670676
 0.0608689 0.0529376 0.04547315 0.03845556 0.02886435]

Размерность после PCA: (589, 3)

PCA sklearn:
Собственные значения:
[2.45450576 1.85655376 1.35296667]

Доля объяснённой дисперсии:
[0.22313689 0.16877761 0.12299697]

Потери при использовании 2 компонент: 0.6080854982741822
Потери при использовании 3 компонент: 0.4850885281608269
```

Первые две главные компоненты сохраняют 39,19% дисперсии исходных данных, при этом потери информации составляют 60,81%. При использовании трёх главных компонент сохраняется уже 51,49% дисперсии, а потери снижаются до 48,51%.

Таким образом, увеличение количества главных компонент с двух до трёх позволяет сохранить больше информации об исходных данных. Полученные графики позволяют визуально представить распределение объектов выборки в пространстве главных компонент.

Вывод: научилась применять метод PCA для осуществления визуализации данных.